

DAirport Management System

Passenger Service Management Use Cases Documentation

Prepared by:

Betül Kaan

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *Viewing Flight Schedules*
- ✓ *Uploading Flight Documents*
- ✓ *Online Check-In*
- ✓ *Boarding Pass Download*
- ✓ *Checking Gate Information*
- ✓ *Tracking Baggage Status*
- ✓ *Receiving Flight Notification*
- ✓ *Each use case is defined with:*

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

UC Name	<i>UC-01: View Flight Schedules</i>
Summary	<i>A passenger views available flight schedules including departure/arrival times and flight status.</i>
Dependency	<i>None</i>
Actors	<i>Primary: Passenger</i>
Preconditions	<i>The passenger has access to the system (web). The system has up-to-date flight data.</i>
Description of the Main Sequence	<i>Step 1: Passenger opens the system and navigates to the flight schedules page.</i> <i>Step 2: Passenger enters search criteria (date, destination, or flight number).</i> <i>Step 3: System retrieves and displays matching flight schedules.</i> <i>Step 4: Passenger views the flight details.</i>
Description of the Alternative Sequence	<i>Step 1: Passenger enters invalid or incomplete search criteria.</i> <i>Step 2: System displays an error message prompting the passenger to correct the input.</i> <i>Step 3: Passenger re-enters valid criteria and the system displays results.</i>
Non functional requirements	<i>The system must process the search and return results within 2 seconds. The interface must be accessible and usable on both desktop and mobile devices.</i>
Postconditions	<i>Flight schedule information is displayed to the passenger.</i>

UC Name	<i>UC-02: Upload Travel Documents</i>
Summary	<i>A passenger uploads required travel documents (e.g., passport, visa) to the system for verification purposes.</i>
Dependency	<i>UC-01 (Passenger may need to view flight info before uploading documents)</i>
Actors	<i>Primary: Passenger</i>
Preconditions	<i>The passenger has an active booking reference. The passenger has digital copies of their documents.</i>
Description of the Main Sequence	<i>Step 1: Passenger logs into the system and navigates to the document upload section. As</i> <i>Step 2: Passenger selects the document type (e.g., passport, visa).</i> <i>Step 3: Passenger uploads the document file.</i>

	Step 4: System validates the file format and size. Step 5: System confirms successful upload and stores the document securely.
Description of the Alternative Sequence	Step 1: Passenger uploads a file in an unsupported format or exceeding the size limit. Step 2: System displays an error message specifying the accepted formats and size. Step 3: Passenger uploads a valid file and the system confirms the upload.
Non functional requirements	Uploaded documents must be encrypted and stored securely. The system must protect personal data in compliance with data protection regulations.
Postconditions	Travel documents are stored in the system and linked to the passenger's profile.

UC Name	UC-03: Perform Online Check-In
Summary	A passenger completes the check-in process online through the airport management system prior to their flight.
Dependency	UC-02: Upload Travel Documents
Actors	Primary: Passenger
Preconditions	The passenger has a valid booking reference. The check-in window for the flight is open. Travel documents have been uploaded.
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the online check-in section. Step 2: Passenger enters their booking reference or flight details. Step 3: System retrieves and displays the passenger's flight information. Step 4: Passenger confirms personal and travel details. Step 5: System processes the check-in and confirms the passenger's checked-in status.
Description of the Alternative Sequence	Step 1: Passenger attempts check-in outside the allowed time window. Step 2: System displays a message indicating that check-in is not yet open or has closed. Step 3: Passenger is redirected to contact customer support or the airline directly.
Non functional requirements	The check-in process must be completable within 3 clicks from the homepage. Page load times must not exceed 3 seconds.
Postconditions	The passenger is marked as checked in and a boarding pass becomes available for download.

UC Name	<i>UC-04: Download Boarding Pass</i>
Summary	<i>A passenger downloads their boarding pass after completing online check-in.</i>
Dependency	<i>UC-03: Perform Online Check-In</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has successfully completed online check-in. The system has generated a boarding pass for the passenger.</i>
Description of the Main Sequence	<i>Step 1: Passenger logs in and navigates to the boarding pass section.</i> <i>Step 2: System retrieves the passenger's boarding pass.</i> <i>Step 3: Passenger selects the download option.</i> <i>Step 4: System generates and downloads the boarding pass as a PDF or image file.</i>
Description of the Alternative Sequence	<i>Step 1: Passenger attempts to download a boarding pass before completing check-in.</i> <i>Step 2: System displays a message indicating that check-in must be completed first.</i> <i>Step 3: Passenger is redirected to the check-in page (UC-03).</i>
Non functional requirements	<i>The boarding pass must be generated and ready for download within 3 seconds. The downloaded file must be compatible with standard mobile and desktop devices.</i>
Postconditions	<i>The passenger has a downloaded boarding pass ready for use at the airport.</i>

UC Name	<i>UC-05: Check Assigned Gate Information</i>
Summary	<i>A passenger checks which gate their flight is departing from.</i>
Dependency	<i>UC-01: View Flight Schedules</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has a valid flight on the day of travel. A gate has been assigned to the flight by airport staff.</i>
Description of the Main Sequence	<i>Step 1: Passenger logs in or accesses the public flight information page.</i> <i>Step 2: Passenger searches for their flight by number or destination.</i> <i>Step 3: System retrieves and displays the assigned gate information.</i>

	Step 4: Passenger views the gate number and terminal details.
Description of the Alternative Sequence	Step 1: Passenger searches for a flight that has not yet been assigned a gate. Step 2: System displays a message indicating the gate assignment is pending. Step 3: Passenger is advised to check back later or enable flight notifications (UC-07).
Non functional requirements	Gate information must be current and updated in real time. The system must display results within 2 seconds of the search.
Postconditions	The passenger is informed of their assigned departure gate.

UC Name	UC-06: Track Baggage Status
Summary	A passenger tracks the current status and location of their checked baggage.
Dependency	UC-03: Perform Online Check-In
Actors	Primary: Passenger
Preconditions	The passenger has checked in and their baggage has been registered in the system by airport staff.
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the baggage tracking section. Step 2: Passenger enters their booking reference or baggage tag number. Step 3: System retrieves and displays the current baggage status and location. Step 4: Passenger reviews the baggage tracking information.
Description of the Alternative Sequence	Step 1: Passenger enters an invalid or unrecognized baggage tag number. Step 2: System displays an error message and prompts the passenger to re-enter or contact support. Step 3: Passenger contacts customer support (links to UC in Customer Support module).
Non functional requirements	Baggage status must reflect real-time updates. The system must handle simultaneous tracking requests from multiple passengers without performance degradation.
Postconditions	The passenger is informed of the current status and location of their baggage.

UC Name	UC-07: Receive Flight Notifications
Summary	A passenger opts in to receive real-time notifications about their flight, including delays, gate changes, and boarding calls.

Dependency	<i>UC-01: View Flight Schedules</i>
Actors	Primary: <i>Passenger</i>
Preconditions	<i>The passenger has a valid flight in the system. The passenger has provided a valid contact method (email or phone number).</i>
Description of the Main Sequence	Step 1: Passenger logs in and navigates to the notifications settings section. Step 2: Passenger selects their flight and preferred notification method (email or SMS). Step 3: Passenger confirms notification preferences. Step 4: System registers the passenger for flight alerts. Step 5: System sends real-time notifications when flight status changes (e.g., delay, gate change, boarding).
Description of the Alternative Sequence	<i>Step 1: Passenger provides an invalid email address or phone number.</i> <i>Step 2: System displays a validation error and prompts the passenger to correct the contact information.</i> <i>Step 3: Passenger re-enters valid contact details and confirms.</i>
Non functional requirements	<i>Notifications must be delivered within a reasonable time of a status change. The system must operate 24/7 to ensure passengers receive timely alerts at any hour.</i>
Postconditions	<i>The passenger receives notifications for any updates related to their flight.</i>

Airport Management System
Flight Operations Management Use Cases Documentation

Prepared by:

Albina Blloshmi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of Baggage Management Use Cases for the Airport Management System.

It includes:

- ✓ *Baggage check-in and registration*
- ✓ *Baggage tracking and real-time status updates*
- ✓ *Handling lost and delayed baggage*
- ✓ *Baggage delivery to passengers*
- ✓ *Baggage routing and transfer between flights*
- ✓ *Security screening of baggage*
- ✓ *Each use case is defined with:*
 - ✓ *Actors*
 - ✓ *Preconditions*
 - ✓ *Main and Alternative Flows*
 - ✓ *Non-functional Requirements*
 - ✓ *Postconditions*

The goal of this document is to clearly model and describe baggage management processes involving both passengers and airport staff in a professional and structured way.

UC Name	UC-08: Claim Baggage at Destination
Summary	A passenger collects their baggage upon arrival at the destination airport.
Dependency	UC-01: Check-In Baggage (external dependency from earlier module)
Actors	Primary: Passenger Airport Staff
Preconditions	Flight has arrived; Baggage has been delivered to the carousel.
Description of the Main	Step 1: Passenger goes to the baggage claim area. Step 2: System displays carousel number. Step 3: Passenger identifies their baggage. Step 4: Passenger collects baggage.
	Step 2: Passenger enters search criteria (date, destination, or flight number). Step 3: System retrieves and displays matching flight schedules. Step 4: Passenger views the flight details.
Description of the Alternative Sequence	Step 1: Passenger cannot find baggage. Step 2: Passenger reports missing baggage to staff.
Non functional requirements	Clear display systems; Minimal waiting time
Postconditions	Passenger successfully retrieves baggage
UC Name	UC-09: Track Baggage in Real-Time
Summary	Passenger monitors baggage location throughout the journey in real time.

Dependency	UC-08: Claim Baggage at Destination (or None if used independently)
Actors	<i>Primary: Passenger</i>
Preconditions	Baggage has been checked in; Tracking system is active.
Description of the Main Sequence	Step 1: Passenger logs into the system.Step 2: Passenger selects baggage tracking option.Step 3: System retrieves real-time data.Step 4: System displays current baggage location.
Description of the Alternative Sequence	Step 1: Tracking data unavailable.Step 2: System shows last known location. <i>Step 3: Passenger re-enters valid criteria and the system displays results.</i>
Non functional requirements	Real-time updates; High accuracy
Postconditions	Passenger is informed of baggage location

UC Name	UC-10: Report Damaged Baggage
Summary	Passenger reports damage to baggage after arrival.
Dependency	UC-08: Claim Baggage at Destination
Actors	Primary: Passenger; Secondary:Airport Staff
Preconditions	Passenger has received baggage

Description of the Main Sequence	Step 1: Passenger inspects baggage.Step 2: Passenger reports damage via system or desk.Step 3: System records complaint.Step 4: Staff verifies damage.Step 5: System processes claim.
Description of the Alternative Sequence	Step 1: Claim rejected due to invalid proof.Step 2: System notifies passenger.
Non functional requirements	Secure claim handling; Quick processing.
Postconditions	Damage report is recorded and processed.

UC Name	UC-11: Handle Delayed Baggage
Summary	System manages cases where baggage is delayed.
Dependency	UC-09: Track Baggage in Real-Time
Actors	Primary: System; Secondary: Staff
Preconditions	Baggage is not delivered on time.
Description of the Main Sequence	Step 1: System detects delay.Step 2: System updates baggage status.Step 3: Passenger is notified.Step 4: Staff arranges delivery.
Description of the Alternative Sequence	Step 1: Baggage cannot be located.Step 2: Lost baggage procedure initiated.
Non functional requirements	Timely notifications; Reliable tracking.

<i>Postconditions</i>	Passenger is informed and baggage is processed for delivery
------------------------------	---

<i>UC Name</i>	UC-12: Deliver Baggage to Passenger
<i>Summary</i>	Baggage is delivered to the passenger after delay or rerouting.
<i>Dependency</i>	UC-11: Handle Delayed Baggage
<i>Actors</i>	Primary: Staff Secondary: Passenger
<i>Preconditions</i>	Baggage has been located.
<i>Description of the Main Sequence</i>	Step 1: Staff schedules delivery Step 2: System updates delivery status Step 3: Baggage is transported to passenger location Step 4: Passenger confirms receipt
<i>Description of the Alternative Sequence</i>	Step 1: Delivery fails Step 2: System reschedules delivery Step 3: Staff is notified to attempt redelivery
<i>Non functional requirements</i>	Timely delivery; Secure handling.

<i>Postconditions</i>	Passenger receives baggage.
------------------------------	-----------------------------

<i>UC Name</i>	UC-13: Update Baggage Information
<i>Summary</i>	Staff updates baggage details in the system.
<i>Dependency</i>	UC-11: Handle Delayed Baggage
<i>Actors</i>	Primary: Staff
<i>Preconditions</i>	Baggage exists in the system.
<i>Description of the Main Sequence</i>	Step 1: Staff accesses baggage record.Step 2: Staff updates details.Step 3: System saves changes.Step 4: System reflects updated info.
<i>Description of the Alternative Sequence</i>	Step 1: Invalid data entered.Step 2: System displays error message.
<i>Non functional requirements</i>	Data accuracy; Secure access.
<i>Postconditions</i>	Baggage information is updated.

Airport Management System
Flight Operations Management Use Cases Documentation

Prepared by:

Alvi Elezi

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a set of Flight Operations Management Use Cases for the Airport Management System.

It includes:

- ✓ *Flight status updates*
- ✓ *Flight delay management*
- ✓ *Departure time updates*
- ✓ *Arrival time updates*
- ✓ *Coordination between boarding processes*
 - ✓ *Flight operations monitoring*
 - ✓ *Each use case is defined with:*
 - ✓ *Actors*
 - ✓ *Preconditions*
 - ✓ *Main and alternative flows*
 - ✓ *Non-functional requirements*
 - ✓ *Postconditions*

The goal of this document is to model and describe the management of flight operations by airport staff and the behavior of the system in a structured way.

UC Name	<i>FO_01 Update Flight Status</i>
Summary	<i>Allows airport staff to update the current status of a flight (e.g., on time, delayed, boarding, departed).</i>
Dependency	<i>Depends on existing flight records in the system.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Staff member is authenticated</i> · <i>Flight exists in the system</i> · <i>Staff has permission to update flight data</i>

Description of the Main Sequence	• <i>Step 1: Airport staff logs into the system.</i> • <i>Step 2: Staff accesses the flight operations dashboard.</i> • <i>Step 3: Staff selects a specific flight.</i> • <i>Step 4: System displays current flight details.</i> • <i>Step 5: Staff updates the flight status (e.g., delayed, boarding).</i> • <i>Step 6: System validates the update.</i> • <i>Step 7: System saves the updated flight status.</i> • <i>Step 8: System notifies relevant stakeholders (passengers, displays).</i>
Description of the Alternative Sequence	• <i>Step 1: Staff selects an invalid flight ID.</i> • <i>Step 2: System displays an error message.</i> • <i>Step 3: If system validation fails, update is rejected and staff must re-enter valid data.</i>
Nonfunctional requirements	· <i>Updates must propagate in real time</i> · <i>System response time must be under 2 seconds</i> · <i>Data consistency must be maintained across modules</i>
Postconditions	<i>Flight status is updated and visible across the system.</i>

UC Name	<i>FO_02 Manage Flight Delays</i>
Summary	<i>Allows airport staff to handle flight delays and update delay-related information.</i>
Dependency	<i>Extends “Update Flight Status”.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Flight exists in the system</i> · <i>Staff is authorized</i>
Description of the Main Sequence	● <i>Step 1: Staff logs into the system.</i> ● <i>Step 2: Staff selects a flight marked for delay.</i> ● <i>Step 3: Staff inputs delay duration and reason.</i> ● <i>Step 4: System updates the departure/arrival time.</i> ● <i>Step 5: System records delay reason.</i> ● <i>Step 6: System sends notifications to passengers.</i>
Description of the Alternative Sequence	● <i>Step 1: Staff enters invalid delay duration.</i> ● <i>Step 2: System prompts correction before saving.</i>
Nonfunctional requirements	· <i>Notifications must be delivered instantly</i>

	· System must handle concurrent updates without conflict
Postconditions	Flight delay is recorded and updated in the system.

UC Name	FO_03 Update Departure Time
Summary	Allows airport staff to modify the scheduled departure time of a flight.
Dependency	Related to the flight scheduling subsystem.
Actors	Primary Actor: Airport Staff
Preconditions	· Flight exists · Staff has update privileges
Description of the Main Sequence	● Step 1: Staff logs into the system. ● Step 2: Staff navigates to flight scheduling. ● Step 3: Staff selects a flight. ● Step 4: Staff enters new departure time. ● Step 5: System validates time constraints. ● Step 6: System updates the flight schedule.

	• <i>Step 7: System notifies passengers and airport displays.</i>
Description of the Alternative Sequence	• <i>Step 1: Entered time conflicts with runway schedule.</i> • <i>Step 2: System rejects update and suggests correction.</i>
Nonfunctional requirements	· <i>System must validate scheduling conflicts</i> · <i>Updates must reflect immediately</i>
Postconditions	<i>Departure time is updated in the system.</i>

UC Name	<i>FO_04 Update Arrival Time</i>
Summary	<i>Allows airport staff to update the arrival time of a flight.</i>
Dependency	<i>Connected to flight tracking systems.</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Flight exists</i>

	· <i>Staff is authorized</i>
Description of the Main Sequence	● <i>Step 1: Staff logs in.</i> ● <i>Step 2: Staff selects flight record.</i> ● <i>Step 3: Staff enters updated arrival time.</i> ● <i>Step 4: System validates input.</i> ● <i>Step 5: System updates arrival time.</i> ● <i>Step 6: System updates passenger notifications.</i>
Description of the Alternative Sequence	● <i>Step 1: Invalid time entered.</i> ● <i>Step 2: System displays validation error.</i>
Nonfunctional requirements	· <i>High data accuracy required</i> · <i>System must synchronize with external systems if integrated</i>
Postconditions	<i>Arrival time is successfully updated.</i>

UC Name	<i>FO_05 Coordinate Boarding Process</i>
Summary	<i>Coordinate Boarding Process</i>
Dependency	<i>Depends on check-in completion and gate assignment.</i>

Actors	<i>Primary Actor: Airport Staff</i> <i>Secondary Actor: Passenger</i>
Preconditions	· <i>Passengers are checked in</i> · <i>Gate is assigned</i> · <i>Boarding process is initiated</i>
Description of the Main Sequence	● <i>Step 1: Staff accesses boarding control interface.</i> ● <i>Step 2: Staff initiates boarding process.</i> ● <i>Step 3: System displays passenger list.</i> ● <i>Step 4: Passengers present boarding passes.</i> ● <i>Step 5: System scans and validates boarding passes.</i> ● <i>Step 6: System marks passengers as boarded.</i> ● <i>Step 7: Staff monitors boarding progress.</i> ● <i>Step 8: Boarding is closed when complete.</i>
Description of the Alternative Sequence	● <i>Step 1: Invalid boarding pass scanned.</i> ● <i>Step 2: System denies boarding and alerts staff.</i>
Nonfunctional requirements	· <i>Real-time passenger validation</i> · <i>High system availability during peak operations</i>
Postconditions	<i>Passenger boarding is completed and recorded.</i>

UC Name	<i>FO_06 Monitor Flight Operations</i>
Summary	<i>Allows airport staff to monitor real-time flight activities and operational status.</i>
Dependency	<i>Integrates with multiple modules (flight, baggage, resources).</i>
Actors	<i>Primary Actor: Airport Staff</i>
Preconditions	· <i>Staff is logged into system</i> · <i>System is operational</i>
Description of the Main Sequence	● <i>Step 1: Staff logs into dashboard.</i> ● <i>Step 2: System displays all active flights.</i> ● <i>Step 3: Staff views real-time updates (status, delays, gates).</i> ● <i>Step 4: Staff identifies issues or anomalies.</i> ● <i>Step 5: Staff takes corrective actions if necessary.</i>
Description of the Alternative Sequence	● <i>Step 1: System fails to retrieve real-time data.</i> ● <i>Step 2: System displays last known status with warning.</i>

Nonfunctional requirements	· <i>Dashboard must refresh in real time</i> · <i>System must support high data throughput</i>
Postconditions	<i>Flight operations are continuously monitored.</i>

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Eneida Balliu

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *User Account Management*
- ✓ *Airport Information Updates*
- ✓ *System Monitoring and Usage Tracking*
- ✓ *Permission and Access Control*
- ✓ *System Maintenance Operations*
- ✓ *System Activity Reporting*

✓ Each use case is defined with:

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

<i>UC Name</i>	UC-40 Manage User Accounts
<i>Summary</i>	This use case describes how the system administrator creates, updates, deletes, and manages user accounts within the system.
<i>Dependency</i>	None
<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has the necessary permissions to manage user accounts. ✓ The system is operational and connected to the database.

<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the user account management module. Step 2: The system displays a list of existing user accounts. Step 3: The administrator selects an action (create, update, or delete account). Step 4: If creating a new account, the administrator enters user details (name, role, email, credentials) . Administrator assigns the user to an airport-specific role (for example: Baggage Handler, Finance Staff, Customer Support, Security Officer). Step 5: If updating, the administrator modifies the required user information. Step 6: If deleting, the administrator selects the account to remove. Step 7: The system validates the entered information. Step 8: The system saves the changes in the database. Step 9: The system confirms the successful operation.
<i>Description of the Alternative Sequence</i>	Step 1: If the administrator enters invalid or incomplete data, the system displays an error message and requests correction. Step 2: If the selected user account does not exist, the system notifies the administrator. Step 3: If there is a system/database failure, the system displays an error and cancels the operation. Step 4: If the administrator cancels the operation, no changes are saved.

<i>Non functional requirements</i>	✓ The system must ensure secure authentication before allowing access to account management. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must process account updates within acceptable response time. ✓ The system must log all account management activities for auditing purposes. ✓ The system must protect user data and ensure privacy.
<i>Postconditions</i>	✓ User account information is successfully created, updated, or deleted. ✓ The system database is updated accordingly. ✓ All changes are recorded in the system logs.

<i>UC Name</i>	<i>UC-41</i> Update Airport Information
<i>Summary</i>	This use case describes how the system administrator updates airport-related information such as facilities, schedules, and general airport data. Admin updates static airport information, such as terminal maps, shop opening hours, runway maintenance statuses, or general airport announcements.
<i>Dependency</i>	None

<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to update airport information. ✓ The system database is available.
<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the airport information management module. Step 2: The system displays current airport information. Step 3: The administrator selects the information to update. Step 4: The administrator edits the required details (e.g., facilities, schedules, announcements). Step 5: The administrator submits the updated information. Step 6: The system validates the entered data. Step 7: The system saves the updated information in the database. Step 8: The system confirms that the update was successful.
<i>Description of the Alternative Sequence</i>	Step 1: If invalid or incomplete data is entered, the system displays an error message and requests correction. Step 2: If the selected information does not exist, the system notifies the administrator. Step 3: If a system or database error occurs, the system cancels the operation and displays an error message.

	Step 4: If the administrator cancels the update, no changes are saved.
Non functional requirements	✓ The system must ensure secure access through authentication. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must process updates within acceptable response time. ✓ The system must ensure data accuracy and consistency. ✓ The system must log all updates for auditing purposes.
Postconditions	✓ Airport information is successfully updated in the system. ✓ The database reflects the latest information. ✓ The update activity is recorded in system logs.

UC Name	UC-42 Monitor System Usage
Summary	This use case describes how the system administrator monitors system activity, usage statistics, and overall system performance.
Dependency	None

<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to access system monitoring tools. ✓ The system is operational and collecting usage data.
<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the system monitoring module. Step 2: The system displays system usage data (active users, transactions, system load). Step 3: The administrator selects specific metrics or reports to view. Step 4: The system retrieves and displays the requested information. Step 5: The administrator reviews system activity and performance indicators. Step 6: The administrator may filter or refine the displayed data. Step 7: The system updates the displayed information based on the selected filters.

<i>Description of the Alternative Sequence</i>	Step 1: If the system cannot retrieve usage data, an error message is displayed. Step 2: If there is a system failure or delay, the system notifies the administrator. Step 3: If no data is available for the selected criteria, the system informs the administrator. Step 4: If the administrator cancels the operation, the process ends without changes.
<i>Non functional requirements</i>	✓ The system must provide real-time system monitoring data. ✓ The system must ensure secure access through authentication. ✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must display data within acceptable response time. ✓ The system must ensure accuracy and reliability of system usage data.
<i>Postconditions</i>	✓ The system administrator has successfully viewed system usage information. ✓ No changes are made to system data. ✓ System monitoring activities may be logged for auditing purposes.

<i>UC Name</i>	UC-43 Manage System Permissions
<i>Summary</i>	This use case describes how the system administrator assigns, updates, and controls user access rights and permissions within the system.
<i>Dependency</i>	UC-40 Manage User Accounts (Can not manage permissions if users do not exist.)
<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to manage system roles and access rights. ✓ User accounts already exist in the system.
<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the permission management module. Step 2: The system displays a list of users and their current roles/permissions. Step 3: The administrator selects a user account. Step 4: The administrator assigns or modifies user roles and permissions. Step 5: The administrator confirms the changes. Step 6: The system validates the permission settings.

	Step 7: The system updates the permissions in the database. Step 8: The system confirms that the permissions have been successfully updated.
<i>Description of the Alternative Sequence</i>	Step 1: If invalid or conflicting permissions are assigned, the system displays an error message. Step 2: If the selected user does not exist, the system notifies the administrator. Step 3: If a system or database error occurs, the system cancels the operation and displays an error message. Step 4: If the administrator cancels the operation, no changes are saved.
<i>Non functional requirements</i>	✓ The system must enforce Role-Based Access Control (RBAC). ✓ The system must ensure secure authentication before permission changes. ✓ The system must log all permission changes for auditing purposes. ✓ The system must ensure data integrity and prevent unauthorized access. ✓ The system must process updates within acceptable response time.
<i>Postconditions</i>	✓ User permissions are successfully updated. ✓ The system database reflects the new access rights. ✓ All changes are recorded in system logs.

<i>UC Name</i>	UC-44 Perform System Maintenance
<i>Summary</i>	This use case describes how the system administrator performs maintenance operations including data backup, data recovery, and system configuration updates.
<i>Dependency</i>	None
<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to perform maintenance operations. ✓ The system is operational and connected to the database

<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the system maintenance module. Step 2: The system displays available maintenance operations (data backup, data recovery, configuration updates). Step 3: The administrator selects a maintenance operation. Step 4: • If backup is selected, the system creates a backup of system data. • If recovery is selected, the administrator selects a backup to restore. • If configuration update is selected, the administrator modifies system settings. Step 5: The administrator confirms the operation. Step 6: The system validates the request. Step 7: The system executes the selected operation. Step 8: The system saves changes and updates the database if necessary. Step 9: The system confirms successful completion.
<i>Description of the Alternative Sequence</i>	Step 1: If invalid input or configuration is provided, the system displays an error message. Step 2: If backup or recovery fails, the system notifies the administrator. Step 3: If a system or database error occurs, the operation is cancelled and an error is displayed. Step 4: If the administrator cancels the operation, no changes are applied.

<i>Non functional requirements</i>	✓ The system must ensure secure access through authentication. ✓ The system must ensure data integrity during backup and recovery operations. ✓ The system must perform maintenance operations within acceptable response time. ✓ The system must log all maintenance activities for auditing purposes. ✓ The system must ensure system availability and minimize downtime.
<i>Postconditions</i>	✓ Maintenance operations are successfully completed. ✓ System data is backed up, restored, or updated accordingly. ✓ All maintenance activities are recorded in system logs.

<i>UC Name</i>	UC-45 Generate System Activity Reports
<i>Summary</i>	This use case describes how the system administrator generates reports on system activity, including user actions, system usage, and performance data.
<i>Dependency</i>	UC_42 Monitor System Usage

<i>Actors</i>	Primary Actor: System Administrator Secondary Actors: None
<i>Preconditions</i>	✓ The system administrator is logged into the system. ✓ The administrator has permission to generate reports. ✓ The system contains activity and usage data.
<i>Description of the Main Sequence</i>	Step 1: The system administrator accesses the reporting module. Step 2: The system displays available report types (user activity, system usage, performance reports). Step 3: The administrator selects a report type. Step 4: The administrator defines report criteria (date range, filters). Step 5: The administrator requests report generation. Step 6: The system retrieves relevant data from the database. Step 7: The system generates the report. Step 8: The system displays the report to the administrator. Step 9: The administrator may export or save the report.

<i>Description of the Alternative Sequence</i>	Step 1: If no data is available for the selected criteria, the system notifies the administrator. Step 2: If invalid criteria are entered, the system displays an error message. Step 3: If a system or database error occurs, the report generation fails and an error is displayed. Step 4: If the administrator cancels the operation, no report is generated.
<i>Non functional requirements</i>	✓ The system must ensure secure access through authentication. ✓ The system must generate reports within acceptable response time. ✓ The system must ensure accuracy and reliability of report data. ✓ The system must allow exporting reports in standard formats (for example: PDF, CSV). ✓ The system must log report generation activities
<i>Postconditions</i>	✓ The system activity report is successfully generated. ✓ The administrator can view or export the report. ✓ Report generation activity is recorded in system logs.

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ameo Lulaj

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

This document presents a structured set of System Administration Use Cases for the Airport Management System. It includes:

- ✓ *Viewing Flight Schedules*
- ✓ *Uploading Flight Documents*
- ✓ *Online Check-In*
- ✓ *Boarding Pass Download*
- ✓ *Checking Gate Information*
- ✓ *Tracking Baggage Status*
- ✓ *Receiving Flight Notification*
- ✓ *Each use case is defined with:*

1. Actors

2. Preconditions

3. Main and Alternative Flows

4. Non-functional Requirements

5. Postconditions

The goal of this document is to clearly model and describe administrative system behavior in a professional and structured way.

UC-ARM-01: Gate Allocation Management

Summary	This use case describes how the system assigns and manages airport gates for arriving and departing flights, ensuring no scheduling conflicts.
Dependency	Flight schedule data must be available.
Actors	Primary Actor: System Administrator / Airport Staff Secondary Actors: None
Preconditions	✓ The administrator is logged into the system. ✓ Flight schedules are available. ✓ Gates are registered in the system.
Main Flow	1. Admin navigates to Gate Management module. 2. System displays available and occupied gates. 3. Admin selects a flight. 4. System suggests available gates. 5. Admin assigns a gate. 6. System updates gate status.
Alternative Flow	• If no gates are available → system displays alert. • Admin can manually override gate assignment.
Postconditions	✓ Gate is successfully assigned. ✓ System reflects updated gate availability.

UC-ARM-02: Runway Scheduling

Summary	This use case describes how runways are scheduled for aircraft takeoff and landing operations.
Dependency	Flight schedule and runway data must be available.
Actors	Primary Actor: System Administrator / Air Traffic Controller Secondary Actors: None
Preconditions	✓ User is logged into the system. ✓ Runways are registered in the system. ✓ Flight schedule exists.
Main Flow	1. Admin accesses runway scheduling module. 2. System displays runway timetable. 3. Admin selects a flight. 4. System suggests available time slots. 5. Admin confirms scheduling. 6. System updates runway allocation.
Alternative Flow	• Emergency flight → system prioritizes runway allocation. • Conflict detected → system prompts rescheduling.
Postconditions	✓ Runway is allocated without conflict. ✓ Schedule is updated successfully.

UC-ARM-03: Staff Allocation

Summary	This use case describes how airport staff are assigned to operational tasks and shifts.
Dependency	Staff database must be available.
Actors	Primary Actor: System Administrator Secondary Actors: Staff
Preconditions	✓ Admin is logged into the system. ✓ Staff details exist in the system.
Main Flow	1. Admin opens staff management module. 2. System displays staff availability. 3. Admin assigns tasks and shifts. 4. System updates staff schedule. 5. Staff receives notification.
Alternative Flow	• If selected staff is unavailable → system suggests alternatives.
Postconditions	✓ Staff are assigned successfully. ✓ Work schedules are updated.

UC-ARM-04: Equipment Management

Summary	This use case describes how airport equipment and assets are tracked, monitored, and maintained.
Dependency	Equipment database must be available.
Actors	Primary Actor: System Administrator Secondary Actors: Maintenance Staff
Preconditions	✓ Admin is logged into the system. ✓ Equipment is registered in the system.
Main Flow	1. Admin accesses equipment management module. 2. System displays equipment list. 3. Admin updates equipment status (available, in use, maintenance). 4. Admin logs maintenance details. 5. System saves updates.
Alternative Flow	• Equipment failure → system generates maintenance alert.
Postconditions	✓ Equipment status is updated. ✓ Maintenance records are stored.

UC-CSM-01: Ticket Management

f	This use case describes how passengers submit and track support queries using a ticketing system.
Dependency	System must support user input and ticket storage.
Actors	Primary Actor: Passenger Secondary Actors: Support Staff
Preconditions	✓ Passenger has access to the system.
Main Flow	1. Passenger opens support page. 2. Passenger submits query via form. 3. System generates a unique ticket ID. 4. Ticket is assigned to support staff. 5. Staff updates ticket status. 6. Passenger tracks ticket progress.
Alternative Flow	• Invalid input → system prompts user to correct details.

Postconditions	✓ Ticket is successfully created. ✓ Issue is tracked and managed.
-----------------------	--

UC-CSM-02: Live Chat Support

Summary	This use case describes how passengers receive real-time assistance through chat support.
Dependency	Chat system must be active and connected.
Actors	Primary Actor: Passenger Secondary Actors: Support Agent, Chatbot
Preconditions	✓ User is online and can access the chat feature.
Main Flow	1. Passenger opens chat widget. 2. Chatbot greets the user. 3. Passenger submits a query. 4. Chatbot responds or transfers to a live agent. 5. Agent resolves the issue. 6. Chat session is stored.
Alternative Flow	• Agent unavailable → chatbot handles query. • Complex issue → system creates

	support ticket automatically.
Postconditions	✓ Query is resolved or escalated. ✓ Chat history is saved.

***Airport Management System
System Administration Use Cases Documentation***

***Prepared by:**
Andri Mucllari*

***Course:**
Software Modeling and Design*

***Date:**
20 March 2026*

Document Overview

*This document presents a structured set of use cases for **Finance & Billing Management** and **Human Resource Management** in the Airport Management System. It includes:*

- *Passenger Payment Processing*
- *Airline Billing and Invoicing*
- *Financial Transaction Tracking*
- *Employee Record Management*
- *Staff Attendance Management*
- *Payroll Processing*

Each use case is defined with:

- *Actors*
- *Preconditions*
- *Main and Alternative Flows*
- *Non-functional Requirements*
- *Postconditions*

The purpose of this document is to clearly describe financial and human resource related system operations in a structured and professional manner.

UC Name	UC_50 Process Airport Service Payment
Summary	This use case describes how passengers pay for airport services such as parking, lounge access, or baggage handling.
Dependency	None
Actors	Primary Actor: Passenger Secondary Actor: Payment System
Preconditions	❖ The passenger is present at the airport or using airport services ❖ The system is operational. ❖ Payment service is available
Description of the Main Sequence	Step 1: The passenger selects an airport service. Step 2: The system displays the service fee. Step 3: The passenger provides payment details. Step 4: The system validates the payment information. Step 5: The system processes the payment. Step 6: The system records the transaction. Step 7: The system generates a receipt. Step 8: The system confirms successful payment.
Description of the Alternative Sequence	Step 1: If payment details are invalid, the system displays an error. Step 2: If payment fails, the system notifies the passenger. Step 3: If system error occurs, the transaction is cancelled. Step 4: If the passenger cancels, no payment is processed.
Non functional requirements	➤ Secure payment processing must be ensured. ➤ Transactions must be processed quickly. ➤ All payments must be logged. ➤ The system must ensure data privacy.
Postconditions	➤ Payment is successfully completed. ➤ Transaction is stored in the system. ➤ Receipt is generated.

UC Name	UC_51 Generate Airport Service Invoice
Summary	This use case describes how the airport generates invoices for services provided to

	external parties (aircraft parking, ground handling).
<i>Dependency</i>	None
<i>Actors</i>	Primary Actor: Finance Staff Secondary Actor: Billing System/Database
<i>Preconditions</i>	❖ The finance staff is logged in ❖ Service usage data is available
<i>Description of the Main Sequence</i>	Step 1: The finance staff selects a service record Step 2: The system retrieves usage details Step 3: The member confirms billing items Step 4: The system calculates total charges Step 5: The system generates the invoice Step 6: The system stores the invoice Step 7: The system confirms successful generation
<i>Description of the Alternative Sequence</i>	Step 1: If data is missing, the system displays an error Step 2: If invalid input is provided, correction is required Step 3: If system error occurs, invoice generation fails
<i>Non functional requirements</i>	➤ The system must ensure billing accuracy ➤ The system must process invoices efficiently ➤ All invoices must be securely stored
<i>Postconditions</i>	➤ Invoice is generated successfully ➤ Billing data is saved
<i>UC Name</i>	UC_52 Monitor Financial Transactions
<i>Summary</i>	This use case describes how financial activities within the airport are monitored and reviewed.
<i>Dependency</i>	UC_50 Process Airport Service Payment
<i>Actors</i>	Primary Actor: Finance Staff /System Administrator
<i>Preconditions</i>	❖ The user is logged into the system ❖ Financial data exists
<i>Description of the Main Sequence</i>	Step 1: The user accesses the financial monitoring module Step 2: The system displays transaction records

	<i>Step 3: The user filters transactions</i> <i>Step 4: The system updates and displays results</i> <i>Step 5: The user reviews financial data</i>
<i>Description of the Alternative Sequence</i>	<i>Step 1: If no data is available, the system notifies the user</i> <i>Step 2: If system error occurs, data cannot be retrieved</i>
<i>Non functional requirements</i>	➤ The system must provide accurate data ➤ The system must respond quickly ➤ Access must be secure
<i>Postconditions</i>	➤ Financial data is successfully reviewed ➤ No system data is changed

<i>UC Name</i>	<i>UC_53 Manage Airport Staff Records</i>
<i>Summary</i>	This use case describes how HR manages records of airport employees such as security staff, ground crew, and administrative staff.
<i>Dependency</i>	None
<i>Actors</i>	<i>Primary Actor: Human Resource Staff</i>
<i>Preconditions</i>	❖ Human resource staff is logged in ❖ System database is available
<i>Description of the Main Sequence</i>	<i>Step 1: HR accesses staff management module</i> <i>Step 2: System displays employee records</i> <i>Step 3: HR selects create, update, or delete</i> <i>Step 4: HR enters or edits employee details</i> <i>Step 5: System validates the data</i> <i>Step 6: System saves changes</i> <i>Step 7: System confirms operation</i>
<i>Description of the Alternative Sequence</i>	<i>Step 1: Invalid data triggers error message</i> <i>Step 2: Non-existent record triggers notification</i> <i>Step 3: System failure cancels operation</i>

<i>Non functional requirements</i>	➤ Employee data must be secure ➤ Role-based access must be enforced ➤ All changes must be logged
<i>Postconditions</i>	➤ Employee records are updated ➤ Changes are stored in the system

<i>UC Name</i>	<i>UC_54 Manage Staff Attendance</i>
<i>Summary</i>	This use case describes how airport staff attendance is recorded and monitored.
<i>Dependency</i>	UC_53 Manage Airport Staff Records
<i>Actors</i>	<i>Primary Actor: Employee</i> <i>Secondary Actor: Human Resource Staff</i>
<i>Preconditions</i>	❖ Employee exists in the system ❖ Attendance system is active
<i>Description of the Main Sequence</i>	<i>Step 1: Employee checks in/out</i> <i>Step 2: System records attendance time</i> <i>Step 3: HR accesses attendance records</i> <i>Step 4: System displays attendance logs</i> <i>Step 5: HR reviews attendance</i>
<i>Description of the Alternative Sequence</i>	<i>Step 1: If check-in fails, system shows error</i> <i>Step 2: If attendance data is missing, system notifies HR</i>
<i>Non functional requirements</i>	➤ System must ensure accurate time tracking ➤ Data must be secure ➤ System must be reliable
<i>Postconditions</i>	➤ Attendance is recorded ➤ Data is stored for future use ➤

UC Name	UC_55 Process Payroll
Summary	This use case describes how salaries for airport staff are calculated and processed.
Dependency	UC_54 Manage Staff Attendance
Actors	Primary Actor: Human Resource Staff / Finance Staff
Preconditions	❖ Employee and attendance data are available ❖ Payroll system is accessible
Description of the Main Sequence	Step 1: HR selects payroll period Step 2: System retrieves staff data Step 3: System calculates salaries (including overtime/deductions) Step 4: HR reviews payroll details Step 5: System processes salary payments Step 6: System records payroll information Step 7: System confirms completion
Description of the Alternative Sequence	Step 1: Missing data triggers error Step 2: Calculation error stops process Step 3: System failure cancels payroll
Non functional requirements	➤ Payroll must be accurate ➤ Data must remain confidential ➤ Processing must be efficient ➤
Postconditions	➤ Salaries are successfully processed ➤ Payroll records are stored

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ani Velo

Course:

Software Modeling and Design

Date:

20 March 2026

Document Overview

*This document presents a structured set of **Security Management Use Cases** for the Airport Management System. It includes:*

- Passenger Security Screening
- Restricted Area Access Control
- Restricted Area Access Control
 - Surveillance Monitoring
 - Surveillance Monitoring
- Emergency Lockdown Procedure

Each use case is defined with:

- Actors
- Preconditions
- Main and Alternative Flows
- Non-functional Requirements
 - Postconditions

The purpose of this document is to clearly describe security management related system operations in a structured and professional manner.

<i>UC Name</i>	<i>UC-SEC-01: Passenger Security Screening</i> <i>UC-SEC-02: Restricted Area Access</i> <i>UC-SEC-06: Log Prohibited Items.</i>
<i>Summary</i>	Manages the secure entry of all individuals into the terminal and restricted zones by verifying identities and documenting seized contraband
<i>Dependency</i>	UC-AMS-01 (Flight Manifest Management): Required to validate passenger boarding passes against current flight data. UC-HR-02 (Staff Records Administration): Required to verify staff credentials and role-based access permissions.
<i>Actors</i>	Primary Actor: Security Officer Secondary Actor: Passenger
<i>Preconditions</i>	❖ Passenger or Staff is present at a security checkpoint with valid physical or digital credentials.
<i>Description of the Main Sequence</i>	Step 1: Hardware sensors capture credential data from the user's pass or ID badge. Step 2: Internal logic validates the data against the Access Control List (ACL). Step 3: A signal is transmitted to the electronic gate to unlock while the Officer's monitor updates to "Proceed". Step 4: The system prompts the Security Officer to record physical search results. Step 5: Upon a "Threat Found" flag, the Seizure Module generates a unique Incident ID and digital receipt. Step 6: The system sets the user's global status to "Cleared" and writes the event to the secure log.

<i>Description of the Alternative Sequence</i>	Invalid Credentials: If data validation fails, the system maintains the hardware lock and logs an "Unauthorized Access" warning. Secondary Search: If a threat is confirmed, the system triggers a mandatory manual search workflow for supervisor review.
<i>Non-functional requirements</i>	➤ Authentication and scanning must conclude in < 1 second. ➤ Contraband logs must be immutable for legal evidence.
<i>Postconditions</i>	➤ User is granted or denied access. ➤ All contraband found is legally documented in the central database.

<i>UC Name</i>	<i>UC-SEC-03: Report Security Incident</i> <i>UC-SEC-04: Monitor Surveillance Feeds</i> <i>UC-SEC-06: Emergency Lockdown</i>
<i>Summary</i>	Provides 24/7 visual oversight and a rapid response framework for logging threats and executing facility-wide lockdowns.
<i>Dependency</i>	UC-SYS-08 (Hardware Node Management): Required for the system to communicate with the Video Management Subsystem (VMS) and camera nodes. UC-SYS-09 (Electronic Door Control): Required for the system to transmit "Lock" signals to physical gate and door hardware.
<i>Actors</i>	Primary Actor: Airport Staff Secondary Actor: System Administrator
<i>Preconditions</i>	❖ Security Personnel are logged into the surveillance dashboard with active camera feeds.
<i>Description of the Main Sequence</i>	Step 1: Video data is continuously streamed from camera nodes to the operator's HD dashboard. Step 2: User-generated "Bookmarks" are received and linked to specific video timestamps for 30-day archiving. Step 3: A new database entry is initialized upon request, assigning a unique tracking number and timestamp. Step 4: A 2-Factor Authentication (2FA) challenge is issued to the Supervisor to authorize emergency state changes. Step 5: A "Global Lock" signal is broadcast to all terminal door controllers and runway gates. Step 6: The PA subsystem is triggered to play automated emergency instructions across all zones.

<i>Description of the Alternative Sequence</i>	<i>Hardware Error: If a camera signal is lost, the system automatically opens a technical maintenance ticket.</i> <i>Authorization Failure: If 2FA verification fails, the system denies the lockdown command and alerts the Lead Administrator.</i>
<i>Non-functional requirements</i>	➤ Lockdown commands must reach all hardware units in < 2 seconds. ➤ 2-Factor Authentication is mandatory for all emergency state changes.
<i>Postconditions</i>	➤ Incident is logged and prioritized ➤ Facility state is transitioned to "Lockdown" if a critical threat is confirmed

Airport Management System

System Administration Use Cases Documentation

Prepared by:

Ajla Alcani

Course:

Software Modeling and Design

Date:

20 March 2026

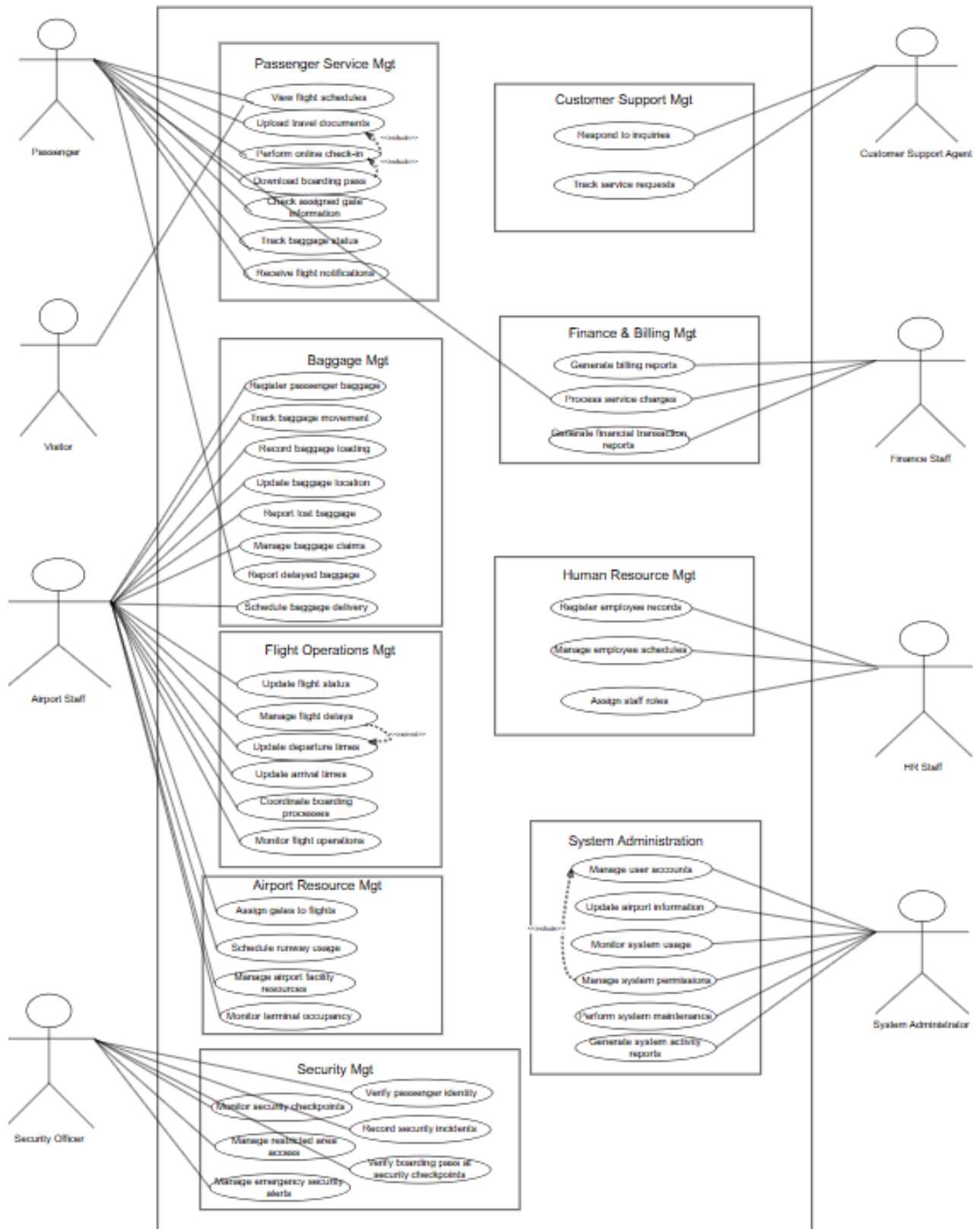
Document Overview

*This document presents the consolidated **Global Use Case Diagram** for the Airport Management System. It includes:*

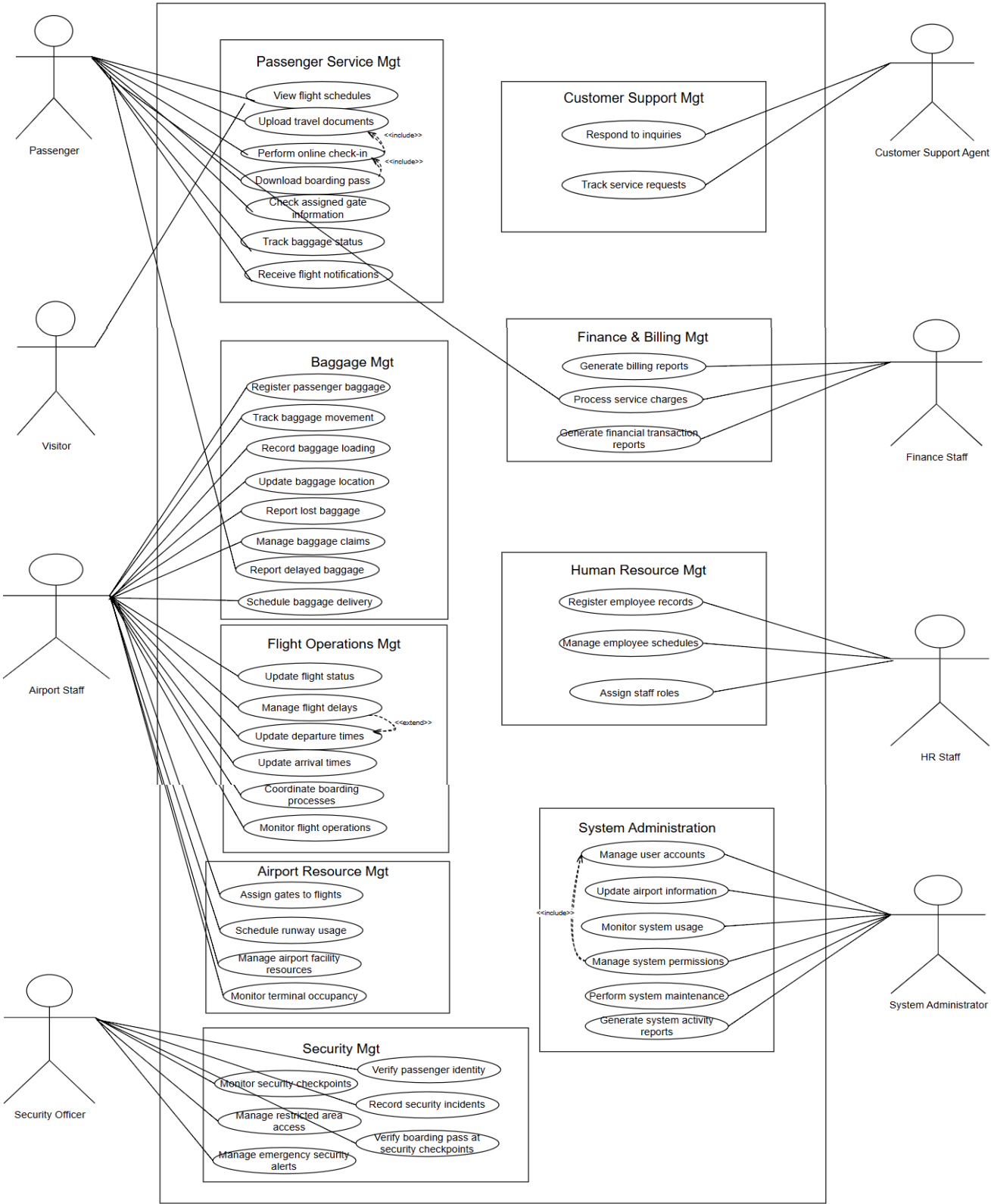
- ✓ Integration of 45 Functional Requirements*
- ✓ Mapping of 8 Primary and Secondary Actors*
- ✓ Cross-Module Logical Dependencies (Includes and Extends)*
- ✓ The diagram is categorized into 9 core system packages:*
 - ✓ Passenger Service Management*
 - ✓ Baggage Management System*
 - ✓ Flight Operations Management*
 - ✓ Security Management System*
 - ✓ System Administration*
 - ✓ Airport Resource Management*
 - ✓ Customer Support Management*
 - ✓ Finance and Billing Management*
 - ✓ Human Resource Management*

The purpose of this document is to clearly model and describe the complete system architecture and actor interactions in a professional and structured way.

Airport Management System



Airport Management System



Global Use Case Diagram Description

The Global Use Case Diagram serves as the architectural visual blueprint for the Airport Management System (AMS). It maps out the complete scope of the software's functional requirements, illustrating exactly how different users interact with the system's various services.

To effectively manage the complexity of the system's 45 functional requirements, the diagram is structured using several key UML modeling principles:

- **Modular Organization (Packages):** Rather than presenting a disorganized web of use cases, the system is divided into nine logical subsystems, represented as UML Packages (e.g., Passenger Service Management, Baggage Management, System Administration). This modularity mirrors professional software architecture and ensures the diagram remains readable and highly organized.
- **Role-Based Interactions (Actors):** The diagram identifies eight distinct actors who interact with the system. Solid association lines clearly establish boundaries regarding which actors can trigger which processes. Primary actors (such as Passengers and Airport Staff) drive the core airport operations, while secondary actors (such as HR Staff and System Administrators) manage the back-office and infrastructure capabilities.
- **System Logic and Dependencies:** To accurately model the real-world workflows of the airport, the diagram utilizes specific UML dependencies to show how use cases interact with one another:
 - **<<include>> Relationships:** These are used to demonstrate mandatory prerequisite actions. For example, the *Perform online check-in* use case explicitly includes *Upload travel documents*, as the system requires the documents before the check-in process can be completed.
 - **<<extend>> Relationships:** These indicate optional, conditional, or alternative flows within the system. For instance, *Manage flight delays* is modeled as an extension of *Update flight status*, as a delay is an alternative scenario that extends the base functionality of a standard status update.

Conclusion: Ultimately, this Global Use Case Diagram bridges the gap between the written requirements and the upcoming system design phase. It provides developers and stakeholders with a clear, comprehensive, and logically sound overview of the entire Airport Management System's intended behavior.